

PHOTON: Hierarchical Autoregressive Modeling for Lightspeed and Memory-Efficient Language Generation

Yuma Ichikawa^{1,2}, Naoya Takagi¹, Takumi Nakagawa^{1,3},
Yuzi Kanazawa¹, Kira Sakai^{1,4}

¹Fujitsu Limited, ²RIKEN Center for AIP, ³Institute of Science Tokyo, ⁴Tokai University

Correspondence: ichikawa.yuma@fujitsu.com

Abstract

Transformers operate as *horizontal token-by-token scanners*; at each generation step, attending to an ever-growing sequence of token-level states. This access pattern increases prefill latency and makes long-context decoding more memory-bound, as KV-cache reads and writes dominate inference time over arithmetic operations. We propose **Parallel Hierarchical Operation for Top-down Networks (PHOTON)**, a hierarchical autoregressive model that replaces horizontal scanning with vertical, multi-resolution context scanning. PHOTON maintains a hierarchy of latent streams: a bottom-up encoder compresses tokens into low-rate contextual states, while lightweight top-down decoders reconstruct fine-grained token representations in parallel. We further introduce *recursive generation* that updates only the coarsest latent stream and eliminates bottom-up re-encoding. Experimental results show that PHOTON is superior to competitive Transformer-based language models regarding the throughput-quality trade-off, providing advantages in long-context and multi-query tasks. In particular, this reduces decode-time KV-cache traffic, yielding up to $10 \times$ higher throughput per unit memory.

1 Introduction

Transformer-based language models have achieved remarkable capabilities; however, the inference cost rapidly increases with context length under recent serving workloads (Bahdanau et al., 2014; Vaswani et al., 2017). Even with KV caching, autoregressive Transformers operate as *horizontal token-by-token scanners*; each new token attends to a continually growing flat history of token-level states. The *prefill* stage computes and stores the KV cache for the entire prompt. During *decoding*, throughput becomes increasingly memory-bound as the context grows, since each step repeatedly reads from and updates a large KV cache. As a re-

sult, performance is often limited by memory bandwidth rather than computational capacity. The bottleneck is most pronounced in long-context, multi-query serving.

This raises a simple question: *Must generation remain horizontal token-by-token scanning over a flat history?* The structure of natural language suggests otherwise (Chomsky, 2002; Lambek, 1958; Hauser et al., 2002; Halle, 1973). Natural language is inherently hierarchical: subwords form words, words form sentences, and sentences create documents. Moreover, coherent generation relies on maintaining an evolving discourse state rather than repeatedly revisiting all fine-grained token representations (Mann and Thompson, 1988; Grosz and Sidner, 1986; Grosz et al., 1995). These observations motivate *vertical scanning*, which represents context through compact coarse states and descends to token-level detail only when necessary.

Hierarchical or multi-scale sequence modeling has recently been explored (Pappagari et al., 2019; Han et al., 2021; Dai et al., 2020; Nawrot et al., 2022, 2023; Fleshman and Van Durme, 2023; Mujika, 2023; Yu et al., 2023; Ho et al., 2024). In particular, Block Transformer (Ho et al., 2024) reduces inference-time KV overhead by separating coarse block-level computation from token-level decoding. However, it employs only a single-level hierarchy that largely serves as a block-structured attention mechanism for efficiency, rather than maintaining a persistent multi-level state that is updated across abstractions during inference.

We introduce **Parallel Hierarchical Operation for Top-down Networks (PHOTON)**, a hierarchical autoregressive model that replaces horizontal token-by-token scanning with multi-resolution vertical scanning over contextual states. As illustrated in Figure 1, PHOTON constructs a hierarchy of latent streams via (i) a bottom-up encoder that compresses tokens into low-rate contextual states and (ii) a top-down decoder stack that progressively re-

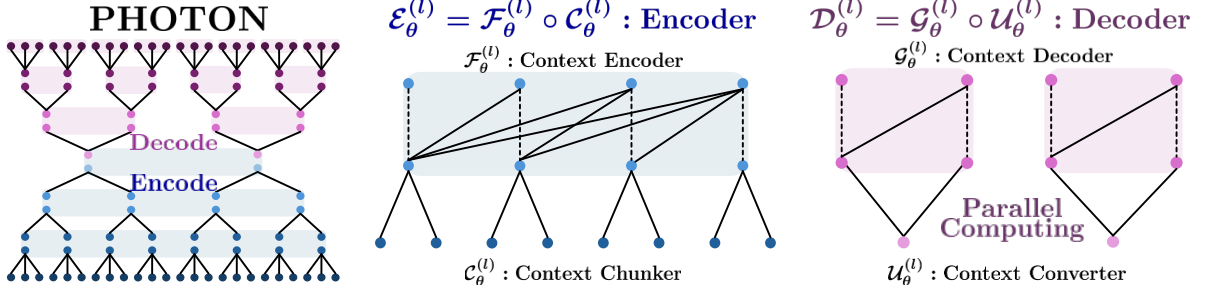


Figure 1: PHOTON overview (left). The hierarchical encoder (middle) compresses token-level states into coarser latent streams via a context chunker and an autoregressive context encoder. The hierarchical decoder (right) expands each coarse latent using a context converter and a chunk-local causal decoder with bounded attention.

constructs finer representations using local autoregressive modules with bounded attention. Since these local decoders operate independently across lower-level contexts, decoding can proceed in parallel across these contexts. PHOTON is trained using standard next-token prediction along with auxiliary objectives that enforce *recursive consistency* at multiple levels by aligning bottom-up summaries with top-down reconstructions. We also propose recursive generation: unlike Block Transformer, PHOTON avoids bottom-up re-encoding of newly generated tokens by directly updating the coarse stream from decoder-side reconstructions. This design retains only the hierarchical decoder on the GPU during decoding, thus reducing both model residency and KV-cache footprint.

Experiments show that PHOTON achieves a better Pareto frontier in terms of throughput per unit memory and quality compared to both vanilla and Block Transformer baselines. In particular, by reducing decode-time KV-cache traffic, PHOTON achieves up to $10^3\times$ higher throughput per unit of memory.

2 PHOTON

2.1 Architecture

This section presents the architecture of PHOTON, a hierarchical autoregressive language model for sequence modeling across multiple resolutions. PHOTON comprises two components: (i) a hierarchical encoder that progressively compresses the input token sequence into coarser latent streams, and (ii) a hierarchical decoder that reconstructs finer-grained streams in a top-down manner using local autoregressive decoders with strictly bounded attention, enabling parallel decoding across independent lower-level contexts. A conceptual overview is provided in Figure 1.

Notation. Let $\mathcal{V}$ denote the vocabulary and $t_{1:T} \in \mathcal{V}^T$ be a sequence of length T . Define $[N] := \{1, \dots, N\}$. Let $X^{(0)} \in \mathbb{R}^{T \times D_0}$ be the embedding matrix, where D_0 is the base hidden dimension. We consider an L -level hierarchy indexed by $l \in \{1, \dots, L\}$. Each level l uses a chunk length $C_l \in \mathbb{N}$ that groups level- $(l-1)$ states into level- l units. Let $M_0 := T$ and $M_l := M_{l-1}/C_l$ denote the number of units at level l . Thus, level l consists of M_l contiguous chunks, each spanning C_l level- $(l-1)$ units. Choose chunk lengths such that T is divisible by $C_{\leq L} := \prod_{k=1}^L C_k$, ensuring all M_l are integers. For each $g \in [M_l]$, define the index set of the g -th level- l chunk as

$$I_g^{(l)} := \{(g-1)C_l + i \mid i \in [C_l]\} \subseteq [M_{l-1}].$$

The corresponding level- $(l-1)$ subtensor is defined as follows:

$$X_{I_g^{(l)}}^{(l-1)} := [X_{(g-1)C_l+1}^{(l-1)}, \dots, X_{gC_l}^{(l-1)}].$$

The sets $\{I_g^{(l)}\}_{g=1}^{M_l}$ partition $[M_{l-1}C_l]$ into contiguous blocks. Since $C_l \geq 1$, we have $M_l \leq M_{l-1}$; thus, $M_0 \geq M_1 \geq \dots \geq M_L$. All learnable parameters are denoted by θ .

2.1.1 Hierarchical Encoder

At each level l , the encoder operates in two stages: (i) it aggregates level- $(l-1)$ representations into chunk-level features, and (ii) it contextualizes these features using an autoregressive context encoder, as shown in Figure 1. Formally,

$$X^{(l)} = \mathcal{E}^{(l)}(X^{(l-1)}) := \mathcal{F}^{(l)} \circ \mathcal{C}^{(l)}(X^{(l-1)}),$$

where the chunker $\mathcal{C}^{(l)}$ and the context encoder $\mathcal{F}^{(l)}$ are defined below.

Context Chunker. The chunker maps each representation $X_{I_g^{(l)}}^{(l-1)}$ to a single chunk representation:

$$A^{(l)} := [A_{1:M_l}^{(l)}] \in \mathbb{R}^{M_l \times D_l}, A_g^{(l)} := \mathcal{C}^{(l)}(X_{I_g^{(l)}}^{(l-1)}).$$

In practice, $\mathcal{C}^{(l)}$ can be implemented by concatenating the representations within a chunk, followed by a linear projection or a one-dimensional convolution. This study utilizes concatenation as a representative instantiation.

Context Encoder. The context encoder captures dependencies among chunk embeddings $\{A_g^{(l)}\}_{g=1}^{M_l}$ using an autoregressive Transformer $X^{(l)} = \mathcal{F}^{(l)}(A^{(l)})$, producing contextualized chunk-level states at level l . Unlike standard Transformers that model sequences token-by-token, our encoder operates over chunk-level contexts, enabling long-range modeling at a coarser temporal resolution.

2.1.2 Hierarchical Decoder

At each level l , the decoder (i) converts each level- l latents into a short conditioning prefix and (ii) reconstructs the level- $(l-1)$ stream using a local autoregressive decoder that operates independently within each chunk, as shown in Figure 1. Formally,

$$\hat{X}^{(l-1)} = \mathcal{D}^{(l)}(\hat{X}^{(l)}) := \mathcal{G}^{(l)} \circ \mathcal{U}^{(l)}(\hat{X}^{(l)}),$$

where the converter $\mathcal{U}^{(l)}$ and the local autoregressive decoder $\mathcal{G}^{(l)}$ are defined below.

Context Converter. To preserve causality, the decoder generates the g -th chunk at level $(l-1)$, conditioned on the previous level- l latent. We introduce a learned starting latent $\hat{X}_0^{(l)} \in \mathbb{R}^{D_l}$ and define $U_{g-1}^{(l)} := \mathcal{U}^{(l)}(\hat{X}_{g-1}^{(l)}) \in \mathbb{R}^{R_l \times D_{l-1}}$ for all $g \in [M_l]$. In our implementation, $\mathcal{U}^{(l)}$ is a one-dimensional convolution that expands a single vector into R_l conditioning vectors.

Context Decoder. Given $U_{g-1}^{(l)}$, the local decoder generates the level- $(l-1)$ latents autoregressively in each chunk. Specifically, for $g \in [M_l]$ and $j \in [C_l]$, the lower-level latents are decoded as follows:

$$\hat{X}_{I_g^{(l)},j}^{(l-1)} = \mathcal{G}^{(l)}\left(U_{g-1}^{(l)}, \hat{X}_{I_g^{(l)},<j}^{(l-1)}; \mathcal{M}_{R_l,j}^{(l-1)}\right),$$

where the indexed context representations are defined as follows:

$$\hat{X}_{I_g^{(l)},j}^{(l-1)} := \hat{X}_{g-1)C_l+j}^{(l-1)}, \hat{X}_{I_g^{(l)},<j}^{(l-1)} := \left[\hat{X}_{I_g^{(l)},i}^{(l-1)}\right]_{i<j}.$$

We implement $\mathcal{G}^{(l)}$ as a causal Transformer applied to the concatenated sequence $[U_{g-1}^{(l)}; \hat{X}_{I_g^{(l)},<j}^{(l-1)}]$ using the standard causal mask $\mathcal{M}_{R_l,j}^{(l-1)} \in \{0, \infty\}^{(R_l+j-1) \times (R_l+j-1)}$.

Under this mask, position j focuses solely on the outputs of the R_l converter and the preceding $(j-1)$ positions within the same chunk. Thus, the attention span is constrained by $R_l + C_l$ and is independent of the global sequence length T . Since each chunk is conditioned only on its corresponding higher-level context via $U_{g-1}^{(l)}$, the chunks can be decoded in parallel across g , resulting in a lightweight local module that enables high-throughput inference.

Overall, PHOTON maps token embeddings through the encoder hierarchy and then back through the decoder hierarchy:

$$\hat{X}^{(0)} = \mathcal{D}^{(1)} \circ \dots \circ \mathcal{D}^{(L)} \circ \mathcal{E}^{(L)} \circ \dots \circ \mathcal{E}^{(1)}(X^{(0)}).$$

Then, $\hat{X}^{(0)}$ is projected onto the vocabulary logits.

akeaway

Hierarchical Encoder compresses the token sequence into *chunk-level latents*, storing long-range context in a compact form.

Hierarchical Decoder decodes lower-level latents *in parallel* with local attention, conditioned on higher-level latents.

2.2 Learning Objective

We train PHOTON using standard next-token prediction with an auxiliary regularizer that promotes consistency among its hierarchical latent streams. Specifically, we minimize the augmented language-modeling loss by including a reconstruction term:

$$\mathcal{L}_{\text{PHOTON}}(\theta; \mathcal{D}) = \mathcal{L}_{\text{token}} + \mathcal{L}_{\text{rec}}, \quad \in \mathbb{R}_+,$$

where $\mathcal{D}$ denotes the training corpus.

Next-Token Loss. Our primary objective is autoregressive maximum likelihood. Given the token-level logits computed from $\hat{X}^{(0)}$, we minimize the negative log-likelihood as follows:

$$\mathcal{L}_{\text{token}} = -\sum_{i=1}^T \log p(t_{i+1} | t_{1:i}).$$

This preserves PHOTON as a standard decoder-only language model at the output level.

Recursive Loss. Next-token supervision alone does not ensure that the model maintains a coherent hierarchy; intermediate latent streams may not be recoverable from the top-down pathway. We introduce an auxiliary objective that explicitly enforces hierarchical consistency by matching each encoder state to its corresponding top-down reconstruction at every level:

$$\mathcal{L}_{\text{rec}} = \sum_{l=1}^L \sum_{g=1}^{M_l} D(\hat{X}_{I_g^{(l)}}^{l-1}, X_{I_g^{(l)}}^{l-1}), \quad (1)$$

where $D(\cdot, \cdot)$ measures the dissimilarity between tensors of the same shape. This study defines D as the cosine distance ($1 - \text{CosineSimilarity}$), computed for each position and averaged. By promoting accurate top-down recovery of bottom-up compressed information, this loss improves the fidelity of the multi-rate latent state. As introduced in section 3.2, it also fosters recursive consistency, enabling faster inference by allowing decoder-side reconstructions to substitute costly bottom-up re-encoding during recursive generation.

2.3 Generation

2.3.1 Inference Bottlenecks in Transformers

Autoregressive Transformers encounter two inference bottlenecks that become more pronounced with longer contexts. During *prefill*, the model processes the entire prompt in parallel, with latency primarily determined by self-attention and feed-forward computations, making this phase predominantly compute-bound. During *decoding*, generation occurs token by token, repeatedly accessing an expanding KV cache to attend to previous states. Because the KV cache grows linearly with context length and batch size, decoding often becomes memory-bound: throughput is limited by KV cache bandwidth rather than arithmetic throughput.

2.3.2 Hierarchical Generation

PHOTON mitigates sequence-length dependence by transferring global computation to low-rate latent streams and restricting token-level decoding to fixed-size local windows. During inference, each chunk is independently decoded in parallel within the same higher-level context; this process is known as hierarchical generation (HierGen).

Computation. At level l , the global encoder processes $M_l \approx T/C_{\leq l}$ latent units, resulting in a prefill cost that scales as $\sum_{l=1}^L \mathcal{O}(M_l^2) =$

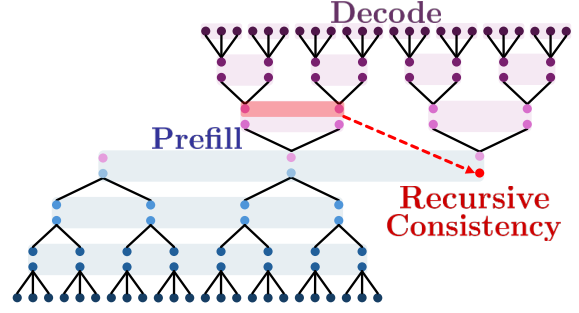


Figure 2: After one-time prefill, recursive generation keeps only the top-level KV cache, decodes meta-contexts top-down, and updates the top-level state via bottleneck summaries.

$\sum_{l=1}^L \mathcal{O}((T/C_{\leq l})^2)$, thereby replacing the vanilla $\mathcal{O}(T^2)$ dependence with a sum over shorter sequences. In contrast, each local decoder attends only within its chunk, with a maximum span of $R_l + C_l$, yielding $\mathcal{O}(1)$ computations per generated token concerning T and facilitating chunk-parallel decoding at a fixed window size.

KV Cache Size. The global encoder stores keys and values for M_l latent units at level l , resulting in a total global KV storage of $\sum_{l=1}^L \mathcal{O}(M_l) = \sum_{l=1}^L \mathcal{O}(T/C_{\leq l})$. Each local decoder, however, requires only the KV cache for the *current* chunk, constrained by $\mathcal{O}(R_l + C_l)$ at level l and is independent of T .

KV Cache Access. At level l , the global context encoder advances once per level- l chunk, i.e., for M_l causal steps over a length- T continuation. Because step g attends to a prefix of length g , the cumulative global KV reads at level l scale as $\sum_{g=1}^{M_l} \mathcal{O}(g) = \mathcal{O}((T/C_{\leq l})^2)$, and the total global KV read traffic is $\sum_{l=1}^L \mathcal{O}((T/C_{\leq l})^2)$. By contrast, each local decoder reads KV only within a bounded window of size $\mathcal{O}(R_l + C_l)$; over a length- T continuation, this produces total local KV reads $\mathcal{O}(T(R_l + C_l)/C_{\leq l-1})$, which grows only linearly in T . Therefore, for large T , global reads dominate; PHOTON reduces total decode-time KV traffic by shortening the global cached sequences at coarser levels and updating them less frequently, while decoding tokens in parallel across chunks using bounded local attention.

2.3.3 Recursive Generation

Hierarchical generation confines token-level autoregression to bounded local windows; however, an encoder-consistent implementation typically re-

quires re-encoding newly generated tokens in a bottom-up manner to refresh the hierarchical states on the encoder-side. This re-encoding triggers additional global-attention updates and increases KV-cache traffic across multiple levels. We introduce recursive generation (RecGen), a decoding schedule that eliminates the need for bottom-up re-encoding. During decoding, only the coarsest stream grows as a global state; it is updated using a summary computed directly from the decoder-side bottleneck reconstruction. We refer to the $C_{\leq L}$ tokens generated by a single top-level step as a meta-context.

Recursive Decoding. Let $KV_{\leq g}^{(L)}$ denote the KV cache of the top-level causal context encoder $\mathcal{F}^{(L)}$ after processing g top-level inputs, and define its streaming update as

$$(X_g^{(L)}, KV_{\leq g}^{(L)}) = \text{ST_P}_{\mathcal{F}^{(L)}}(A_g^{(L)}, KV_{\leq g-1}^{(L)}).$$

After a one-time hierarchical prefill on the prompt, we retain $KV^{(L)}$ and discard encoder-side caches at levels $1, \dots, L-1$. At each meta context step g to $g+1$, we (i) run the top-down decoders conditioned on $X_g^{(L)}$ to sample the next tokens and obtain the bottleneck reconstruction $\hat{X}_{I_{g+1}^{(L)}}^{(L-1)}$, (ii) summarize it using the same chunker $\hat{A}_{g+1}^{(L)} := \mathcal{C}^{(L)}(\hat{X}_{I_{g+1}^{(L)}}^{(L-1)})$, and (iii) update the coarsest stream by

$$(X_{g+1}^{(L)}, KV_{\leq g+1}^{(L)}) = \text{ST_P}_{\mathcal{F}^{(L)}}(\hat{A}_{g+1}^{(L)}, KV_{\leq g}^{(L)}).$$

Therefore, $KV^{(L)}$ is the only context-growing global cache during decoding; all KV caches in the top-down decoders are local, constrained by fixed windows, and can be discarded at chunk boundaries.

Recursive Consistency and Equivalence. RecGen replaces encoder-side re-encoding with a summary computed from the decoder-side bottleneck reconstruction. A sufficient condition for exact equivalence to HierGen with bottom-up re-encoding is bottleneck *recursive consistency*, namely $\hat{X}^{(L-1)} = X^{(L-1)}$. Under this condition, we have $\hat{A}_g^{(L)} = A_g^{(L)}$, meaning both procedures apply the same top-level updates and therefore induce the same distribution over output token sequences. Equation (1) explicitly encourages this consistency, making decoder-derived summaries a reliable substitute for encoder-side re-encoding.

We provide the derivation and additional theoretical results in Appendix A.

Inference Cost. The prefill remains unchanged; we run the hierarchical encoder once on the prompt. All differences arise during decoding. In HierGen, the model maintains and advances global encoder streams across all levels, resulting in cumulative KV reads $\sum_{l=1}^L \mathcal{O}(M_l^2) = \sum_{l=1}^L \mathcal{O}((T/C_{\leq l})^2)$. In contrast, RecGen retains only the top-level cache $KV^{(L)}$ and performs a single global streaming update per meta-context, reducing the global KV footprint from $\sum_{l=1}^L \mathcal{O}(T/C_{\leq l})$ to $\mathcal{O}(T/C_{\leq L})$. As a result, global KV reads collapse to the top-level term $\mathcal{O}((T/C_{\leq L})^2)$. All KV accesses within the top-down decoders are constrained by fixed attention windows, leading to linear increases with T .

akeaway

RecGen speeds up decoding over **HierGen** by keeping global KV only at the top level and skipping bottom-up re-encoding, halving the GPU-resident model footprint.

Memory	HierGen	RecGen
Size	$\sum_l \mathcal{O}(T/C_{\leq l})$	$\mathcal{O}(T/C_{\leq L})$
Access	$\sum_l \mathcal{O}((T/C_{\leq l})^2)$	$\mathcal{O}((T/C_{\leq L})^2)$

3 Experiments

Architecture and Training Configurations. All models are based on LLaMA Transformer architecture (Touvron et al., 2023). We compare PHOTON to vanilla Transformer and block Transformer (Ho et al., 2024). For each PHOTON configuration, we tune the baseline architectures to have approximately the same number of trainable parameters; see the details in Appendix C.2. PHOTON is trained on the Pile (Gao et al., 2020) with a context length of 2048 for one epoch. For the Vanilla and Block Transformer baselines, we align the training compute budget with the corresponding PHOTON model in terms of total FLOPs. All experiments are conducted on NVIDIA DGX H200 GPUs. Additional implementation details are included in Appendix C.

Evaluation. We evaluate (i) inference efficiency and (ii) language modeling quality. Following the empirical protocol of Ho et al. (2024), we measure the per-sample KV-cache memory footprint and throughput (K tokens/s) under two complementary

Table 1: Inference efficiency and language modeling quality for PHOTON and Vanilla/Block Transformer baselines at matched model sizes. Memory is measured per sample (GiB), throughput in K tokens/s, and TPM in K tokens/s/GiB. Quality is evaluated by WikiText perplexity and zero-shot accuracy on HS, SciQ, and ARCe.

Models	TPM		Memory		Throughput		PPL and Zero-shot Accuracy			
	PF $\uparrow$	DE $\uparrow$	PF $\downarrow$	DE $\downarrow$	PF $\uparrow$	DE $\uparrow$	WikiText $\downarrow$	HS $\uparrow$	SciQ $\uparrow$	ARCe $\uparrow$
Vanilla Transformer										
600 M	3.24	7.35	0.275	0.230	0.89	1.69	22.3793	41.24	72.30	43.94
900 M	3.99	6.61	0.298	0.354	1.19	2.34	20.4102	44.72	76.10	47.73
1.2 B	1.21	2.56	0.439	0.390	0.53	1.00	19.6831	45.65	81.50	49.33
Block Transformer										
600 M	562.73	1528.71	0.044	0.031	24.76	47.39	27.2478	37.01	70.30	42.63
900 M	485.74	1386.05	0.054	0.038	26.23	52.67	26.3706	37.06	71.20	43.43
1.2 B	205.00	540.20	0.070	0.051	14.35	27.55	22.8429	41.74	74.90	45.83
PHOTON										
600 M	1262.58	3062.17	0.031	0.023	39.14	70.43	29.9055	35.49	67.70	42.97
900 M	1141.62	2797.04	0.037	0.027	42.24	75.52	26.2325	38.24	69.00	44.32
1.2 B	543.86	1216.67	0.044	0.036	23.93	43.80	23.7863	40.70	69.30	46.25

serving regimes: *prefill-heavy* (PF), with a long prompt and short continuation (2048 input/128 output), and *decode-heavy* (DE), with a short prompt and long continuation (128 input/2048 output). To summarize memory efficiency in multi-query serving, we also report throughput-per-memory (TPM), defined as $\text{TPM} = \text{Throughput} / \text{Memory}$ in K tokens/s/GiB. For quality, we report WikiText perplexity (PPL) (Merity et al., 2017) and zero-shot accuracy on HellaSwag (HS) (Zellers et al., 2019), SciQ (Welbl et al., 2017), and ARC-Easy (ARCe) (Clark et al., 2018).

3.1 Main Results

Throughput-per-Memory Gains. PHOTON primarily enhances inference efficiency by reducing the KV cache footprint and associated memory traffic during serving. This section adopts a representative two-level hierarchy ($L = 2$) with chunk lengths $C_1 = 4$ and $C_2 = 4$; Appendix B.2 varies these chunk lengths and demonstrates that PHOTON maintains the most favorable TPM-quality trade-off across context-length settings. In the main results, we set the recursive reconstruction weight to $\alpha = 0.0$ to isolate gains from the hierarchical architecture and bounded local parallel decoding of PHOTON, which differ from the Block Transformer architecture. Appendix B.1 further studies α and shows that a moderate value, around $\alpha \approx 0.3$, maximizes downstream zero-shot accuracy, supporting the effectiveness of the proposed recursive regularization.

Table 1 presents the KV cache memory and

throughput per-sample under PF and DE settings. Across model scales, PHOTON reduces KV-cache memory while increasing throughput, resulting in substantial gains in TPM. In the PF regime, PHOTON reduces KV-cache memory per sample by $8.9\times$ (600M), $8.1\times$ (900M), and $10.0\times$ (1.2B), while enhancing throughput by $44.0\times$, $35.5\times$, and $45.2\times$, respectively. In the DE regime, PHOTON reduces KV memory by $10.0\times$ (600M), $13.1\times$ (900M), and $10.8\times$ (1.2B), and increases throughput by $41.7\times$, $32.3\times$, and $43.8\times$. As a result, PHOTON achieves substantially higher TPM than a vanilla Transformer in both regimes; for the 1.2B model in DE, TPM increases from 2.56 to 1216.67 K tok/s/GiB, corresponding to a $475\times$ improvement. Overall, these results support our central claim that maintaining a hierarchical state reduces decode-time KV traffic and enables more memory-efficient generation.

TPM vs. Language Modeling Quality. Figure 3 shows the trade-off between TPM and language modeling quality. PHOTON achieves significantly higher TPM with only moderate degradation in WikiText PPL. At 600M, TPM increases from 3.24 K tokens/s/GiB to 1262.58 K tokens/s/GiB in the PF regime, and from 7.35 to 3062.17 in DE regime, while PPL slightly increases from 22.38 to 29.91. For the 1.2B model, TPM rises from 1.21 K tokens/s/GiB to 543.86 K tokens/s/GiB in the PF regime and from 2.56 K tokens/s/GiB to 1216.67 K tokens/s/GiB in DE regime, while PPL moderately increases from 19.68 to 23.79. As a result, PHOTON

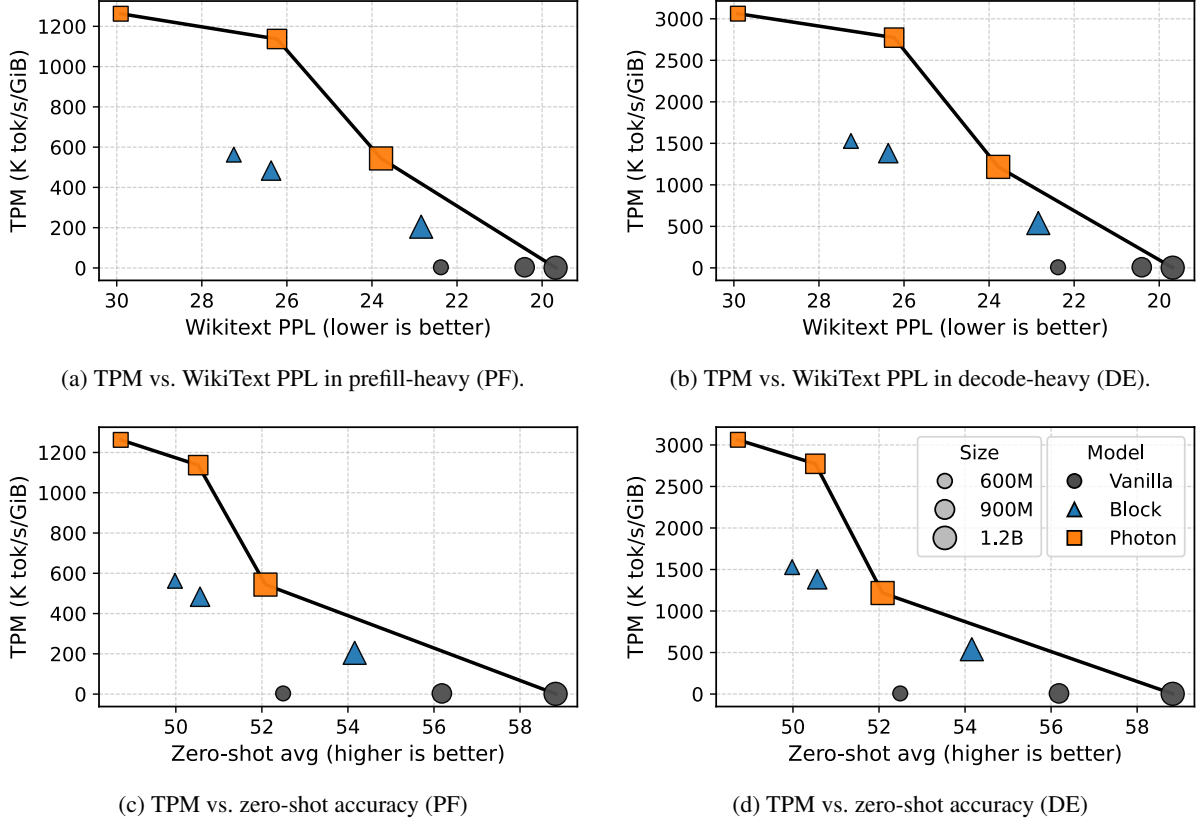


Figure 3: TPM–quality trade-offs under PF and DE regimes. Panels (a,b) plot TPM against WikiText perplexity, and panels (c,d) plot TPM against average zero-shot accuracy over HS, SCiQ, and ARCe. The dotted line denotes the Pareto frontier in each panel. Across both regimes and all metrics, PHOTON consistently yields a more favorable TPM–quality frontier than Vanilla and Block Transformers.

achieves significantly higher TPM than Vanilla models in both regimes. Compared to the Block Transformer, PHOTON achieves higher TPM at both model sizes across both regimes; dominating the Block Transformer concerning the Pareto frontier in Figure 3. This indicates that PHOTON provides a superior TPM-quality operating point, and the saved memory bandwidth can be traded back for quality through test-time scaling within a fixed memory budget.

3.2 Recursive Generation

We evaluate RecGen as a drop-in decoding strategy for the 600M PHOTON model under PF and DE settings. We compare RecGen to HierGen, which eliminates the need for bottom-up re-encoding by updating only the coarsest latent stream and utilizing summaries computed from the decoder-side bottleneck reconstruction. As shown in Table 2, RecGen enhances efficiency while maintaining nearly unchanged generation behavior. Appendix B.3 supports this observation by demonstrating that the recursive loss steadily decreases throughout

Table 2: Inference efficiency gains from RecGen over standard HierGen for 600M PHOTON model. Memory usage per sample is reported in GiB, throughput, and TPM under PF and DE settings.

Metric	Setting	Generation	
		HierGen	RecGen
TPM	PF↑	1262.58	2828.06
	DE↑	3062.17	13642.50
Memory	PF↓	0.031	0.031
	DE↓	0.023	0.012
Throughput	PF↑	39.14	87.67
	DE↑	70.43	163.71

training. In PF regime, RecGen increases throughput from 39.14 to 87.67 K tok/s ($\approx 2.2\times$) while maintaining the same memory footprint, thus improving TPM by $\approx 2.2\times$. In DE regime, RecGen throughput increased from 70.43 to 163.71 K tok/s ($\approx 2.3\times$), while memory consumption decreased from 0.023 to 0.012 GiB ($\approx 1.9\times$), resulting in a $4.5\times$ increase in TPM. Notably, since 600M model

already achieves roughly 7.35 TPM in Table 1, this implies that RecGen can attain a maximum TPM advantage of up to $1,856\times$ in the corresponding setting. Overall, RecGen shifts decoding from global KV-cache traffic to bounded local computation, enhancing its suitability for multi-query workloads under fixed GPU memory constraints.

4 Related Work

PHOTON is associated with (i) hierarchical and multi-scale Transformers, (ii) inference efficiency in KV terms, and (iii) global-local modeling in tokenizer-free language models.

Hierarchical and Multi-scale Transformers. A substantial body of work reduces the effective length of global attention by introducing intermediate representations. For example, hierarchical encoders for long documents (Pappagari et al., 2019), patch-based or segment-based Transformers (Han et al., 2021), and down-sampled or up-sampled architectures such as Funnel and Hourglass (Dai et al., 2020; Nawrot et al., 2022; Zhu and Soricut, 2021). These approaches primarily aim for better representations, improved training stability, or reduced training costs. However, during inference, they typically maintain token-level autoregressive decoding, resulting in a KV cache that grows linearly with context length. In contrast, PHOTON treats hierarchy as an inference primitive; it maintains persistent multi-rate latent streams as global state and confines token-level computation to strictly bounded local refinement.

KV-Efficient Inference. Previous work accelerates long-context inference primarily by modifying attention mechanisms, such as sparse or windowed patterns (Child et al., 2019; Beltagy et al., 2020), and by dynamically retaining a subset of tokens during training or inference (Nawrot et al., 2023; Fu et al., 2025). Although effective, these methods maintain a single token-level timeline: decoding occurs token by token, which limits inference bandwidth due to repeated KV-cache reads and writes. PHOTON addresses the same bottleneck but employs a different design principle: it reduces KV traffic through a persistent hierarchical state. Specifically, PHOTON factorizes generation into an encoder-decoder hierarchy that represents global context as low-rate latent streams and produces tokens with strictly bounded local causal decoders. This structure enables parallel decoding of inde-

pendent chunks conditioned on higher-level latents, thereby shrinking the globally cached sequence and reducing the need for global updates.

Tokenizer-free Models. Tokenizer-free and byte-level language models often adopt global-local hierarchies to model long byte streams tractably, as seen in MEGABYTE (Yu et al., 2023), Space-Byte (Slagle, 2024), and learned segmentation approaches (Zakershahraak and Ghodratnama, 2025; Fleshman and Van Durme, 2023; Mujika, 2023). In these models, the hierarchy induces subword-like units from bytes for a single autoregressive stream: a global module models patches or segments, while a local module reconstructs bytes or characters without maintaining a persistent multi-level state during decoding. PHOTON targets a different objective. Rather than learning subword units from bytes, it introduces higher-level contextual latents to reduce redundant computation and accelerate inference for subword-token language models. This results in a compact encoder-decoder design in which the encoder maintains a persistent coarse context summary, while local decoders use it to refine token-level generation, thereby reducing inference-time memory traffic.

5 Conclusion

We presented PHOTON, a hierarchical autoregressive language model that replaces horizontal *token-by-token scanning* in Transformers with *vertical* multi-resolution context scanning. PHOTON builds a hierarchy of latent streams through (i) a bottom-up encoder that compresses the token sequence into low-rate contextual states and (ii) a top-down decoder stack that reconstructs progressively finer representations using local autoregressive modules with bounded attention, enabling parallel decoding across independent contexts. We further introduced *recursive generation*, updating only the coarsest latent stream during decoding and avoiding bottom-up re-encoding, thus reducing KV-cache growth and memory traffic. Experiments demonstrate that PHOTON consistently improves the TPM–quality trade-off over strong vanilla and Block Transformer baselines, achieving up to $10^3\times$ higher TPM. These results suggest that the persistent hierarchical state and local reconstruction provide a promising direction for scalable, memory-efficient language generation.

Limitations

This work has several limitations. First, we train and evaluate PHOTON using a single pretraining corpus and a relatively small set of downstream benchmarks; broader coverage is needed to confirm whether the observed trends generalize across different data mixtures and task families. Second, our largest model contains 1.2 billion parameters, and we have yet to characterize how the efficiency-quality trade-off of PHOTON performs at larger scales. Third, while we report representative results for certain architectural configurations, we do not include a comprehensive sensitivity analysis of key design and training choices, such as chunk sizes and converter widths. A more exhaustive ablation study would help isolate the contributions of each component and clarify which settings are most robust.

References

- Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. 2014. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.
- Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. 2019. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*.
- Noam Chomsky. 2002. *Syntactic structures*. Walter de Gruyter.
- Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv, abs/1803.05457*.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V. Le, and Ruslan Salakhutdinov. 2020. Funnel-transformer: Filtering out sequential redundancy for efficient language processing. In *Advances in Neural Information Processing Systems*.
- William Fleshman and Benjamin Van Durme. 2023. Toucan: Token-aware character level language modeling. *arXiv preprint arXiv:2311.08620*.
- Zhen Fu and 1 others. 2025. Sliding window attention training for efficient large-context language models. *arXiv preprint arXiv:2502.18845*.
- Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, and 1 others. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.
- Barbara J Grosz, Aravind Joshi, and Scott Weinstein. 1995. Centering: A framework for modeling the local coherence of discourse. *Computational linguistics*, 21(2):203–225.
- Barbara J Grosz and Candace L Sidner. 1986. Attention, intentions, and the structure of discourse. *Computational linguistics*, 12(3):175–204.
- Morris Halle. 1973. Prolegomena to a theory of word formation. *Linguistic inquiry*, 4(1):3–16.
- Kai Han, An Xiao, Enhua Wu, Jianyuan Guo, Chunjing Xu, and Yunhe Wang. 2021. Transformer in transformer. In *Advances in Neural Information Processing Systems*.
- Marc D Hauser, Noam Chomsky, and W Tecumseh Fitch. 2002. The faculty of language: what is it, who has it, and how did it evolve? *science*, 298(5598):1569–1579.
- Namgyu Ho, Sangmin Bae, Taehyeon Kim, Hyunjik Jo, Yireun Kim, Tal Schuster, Adam Fisch, James Thorne, and Se-Young Yun. 2024. Block transformer: Global-to-local language modeling for fast inference. In *Advances in Neural Information Processing Systems*.
- Joachim Lambek. 1958. The mathematics of sentence structure. *The American Mathematical Monthly*, 65(3):154–170.
- William C Mann and Sandra A Thompson. 1988. Rhetorical structure theory: Toward a functional theory of text organization. *Text-interdisciplinary Journal for the Study of Discourse*, 8(3):243–281.
- Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2017. Pointer sentinel mixture models. In *International Conference on Learning Representations*.
- Asier Mujika. 2023. Hierarchical attention encoder-decoder architectures for long-range sequence modeling. *arXiv preprint arXiv:2306.01070*.
- Piotr Nawrot, Jan Chorowski, Adrian Łańcucki, and Edoardo Maria Ponti. 2023. Efficient transformers with dynamic token pooling. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*.
- Piotr Nawrot, Szymon Tworkowski, Michał Tyrolski, Łukasz Kaiser, Yuhuai Wu, Christian Szegedy, and Henryk Michalewski. 2022. Hierarchical transformers are more efficient language models. In *Findings of the Association for Computational Linguistics: NLP*.

Raghavendra Pappagari, Piotr Zelasko, Jesús Villalba, Yishay Carmiel, and Najim Dehak. 2019. Hierarchical transformers for long document classification. In *2019 IEEE automatic speech recognition and understanding workshop (ASRU)*.

Kevin Slagle. 2024. Spacebyte: Towards deleting tokenization from large language modeling. In *Advances in Neural Information Processing Systems*.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, and 1 others. 2023. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*.

Johannes Welbl, Nelson F. Liu, and Matt Gardner. 2017. Crowdsourcing multiple choice science questions. *arXiv, abs/1707.06209*.

Lili Yu, Daniel Simig, Colin Flaherty, Armen Aghajanyan, Luke Zettlemoyer, and Mike Lewis. 2023. Megabyte: Predicting million-byte sequences with multiscale transformers. In *Advances in Neural Information Processing Systems*.

Mehrdad Zakershahraei and Samira Ghodrathnama. 2025. H-net++: Hierarchical dynamic chunking for tokenizer-free language modelling in morphologically-rich languages. *arXiv preprint arXiv:2508.05628*.

Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. 2019. Hellaswag: Can a machine really finish your sentence? *arXiv preprint arXiv:1905.07830*.

Zhenhai Zhu and Radu Soricut. 2021. H-transformer-1d: Fast one-dimensional hierarchical attention for sequences. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics*.

Additional Theoretical Results

1 Preliminaries

Fix the number of hierarchy levels L and recall the cumulative meta context length $B := C_{\leq L} = \prod_{k=1}^L C_k$. For $g \in [L]$, define the token index set of the g -th meta context by

$$J_g := \{(g-1)B + 1, \dots, gB\}.$$

For simplicity, we assume T is divisible by B , i.e., $T = GB$ for an integer G . Throughout, we focus on the coarsest causal context encoder $\mathcal{F}^{(L)}$ and its KV cache. Let $\text{KV}_{\leq g}^{(L)}$ denote the KV cache after processing g top-level inputs. We express a one-step streaming update by a deterministic operator:

$$\left(X_g^{(L)}, \text{KV}_{\leq g}^{(L)}\right) = \text{ST-P}_{\mathcal{F}^{(L)}}\left(A_g^{(L)}, \text{KV}_{\leq g-1}^{(L)}\right), \quad (2)$$

where $A_g^{(L)} \in \mathbb{R}^{D_L}$ is the g -th top-level input summary.

Assumption 1 (Deterministic Streaming Update). *The operator $\text{ST-P}_{\mathcal{F}^{(L)}}$ in Equation (2) is deterministic given its arguments.*

Assumption A.1 holds for standard causal Transformers with KV caching: given an input vector and a cache, the next hidden state and the updated cache are uniquely determined. As an interesting direction for future work, one could relax this assumption by allowing the model to output context *stochastically*, i.e., replacing $\text{ST-P}_{\mathcal{F}^{(L)}}$ with a conditional distribution over updates—potentially enabling richer uncertainty-aware streaming and alternative decoding schedules.

2 Two Decoding Procedures

We compare two inference procedures that use the same top-down decoder stack $\{\mathcal{D}^{(l)}\}_{l=1}^L$ and differ only in their construction of the subsequent top-level input.

Definition 2 (Hierarchical Generation (Hier-Gen)). *Fix a prompt and perform a one-time hierarchical prefill to initialize all encoder-side states and caches. During decoding, at each meta context step $g \rightarrow g+1$, hierarchical generation:*

1. *samples the next meta context tokens $t_{J_{g+1}}$ by running the top-down decoders conditioned on the current top-level state $X_g^{(L)}$;*

2. re-encodes the newly generated tokens bottom-up to obtain the encoder-side bottleneck block $X_{I_{g+1}}^{L-1}$ and its top-level input summary

$$A_{g+1}^{(L)} := \mathcal{C}^{(L)}(X_{I_{g+1}}^{L-1});$$

3. advances the top-level stream via Equation (2) with $A_{g+1}^{(L)}$.

Let p^{Hier} denote the induced distribution over generated continuations.

Definition .3 (Recursive Generation (RecGen)). Fix a prompt and perform a one-time hierarchical prefill, retaining only the top-level cache $\text{KV}^{(L)}$ thereafter. During decoding, at each meta context step $g \rightarrow g+1$, RecGen:

1. samples the next meta context tokens $t_{J_{g+1}}$ by running the same top-down decoders conditioned on $X_g^{(L)}$, and simultaneously obtains the decoder-side bottleneck reconstruction $\hat{X}_{I_{g+1}}^{L-1}$ produced by the top-down stack;
2. forms the next top-level input summary directly from the reconstruction:

$$\hat{A}_{g+1}^{(L)} := \mathcal{C}^{(L)}(\hat{X}_{I_{g+1}}^{L-1});$$

3. advances the top-level stream via Equation (2) with $\hat{A}_{g+1}^{(L)}$.

Let p^{Rec} denote the induced distribution over generated continuations.

.3 Recursive Consistency and Exactness

Definition .4 (Recursive consistency). We say that recursive consistency holds if, for every meta context $g \in [G]$, the reconstruction at the decoder-side bottleneck matches the encoder-side bottleneck state:

$$\hat{X}_{I_g}^{L-1} = X_{I_g}^{L-1}. \quad (3)$$

From Definition A.4, the reconstruction at the decoder-side bottleneck is equal to the state at the encoder-side bottleneck for each meta-context. Since the top-level input summary is computed deterministically from this bottleneck representation, it follows that the top-level inputs obtained through re-encoding and recursion must coincide, yielding the following Lemma A.5.

Lemma .5. Under Definition .4, the top-level inputs produced by re-encoding and recursion agree for all $g \in [G]$:

$$\hat{A}_g^{(L)} = A_g^{(L)}. \quad (4)$$

Proof. By Equation (3) and the definition of the chunker $\mathcal{C}^{(L)}$,

$$\hat{A}_g^{(L)} = \mathcal{C}^{(L)}(\hat{X}_{I_g}^{L-1}) = \mathcal{C}^{(L)}(X_{I_g}^{L-1}) = A_g^{(L)},$$

which is Equation (4). $\square$

Lemma A.5 demonstrates that recursion yields the same top-level inputs as bottom-up re-encoding at each meta-context step. Combined with Assumption A.1, the resulting top-level hidden states and KV caches evolve identically under recursive and exact hierarchical generation. Consequently, both procedures induce the same conditional distribution at each decoding step, directly implying Theorem A.6.

Theorem .6. Assume Assumption .1 and bottleneck recursive consistency (Definition .4). Then RecGen (Definition .3) induces the same distribution over output token sequences as exact hierarchical generation (Definition .2):

$$p^{\text{Rec}} = p^{\text{Hier}}.$$

Proof. We couple the two procedures by using the same random draws for token sampling whenever their conditional distributions match. We prove by induction on g that the top-level state and cache coincide for all $g \in \{0, \dots, G\}$

$$(X_g^{(L)}, \text{KV}_{\leq g}^{(L)})_{\text{Rec}} = (X_g^{(L)}, \text{KV}_{\leq g}^{(L)})_{\text{Hier}}. \quad (5)$$

They coincide at $g = 0$ after the shared prompt prefill. Assume Equation (5) holds at step g . Conditioned on the identical top-level state $X_g^{(L)}$, both procedures run the same top-down decoder stack with the same causal masking, thus defining the same conditional distribution over the next meta context $t_{J_{g+1}}$. Using the coupling, both procedures sample the same realization of $t_{J_{g+1}}$.

By Lemma A.5, the top-level inputs then agree: $\hat{A}_{g+1}^{(L)} = A_{g+1}^{(L)}$. Finally, by Assumption A.1 and the shared cache $\text{KV}_{\leq g}^{(L)}$, the deterministic update Equation (2) yields identical $(X_{g+1}^{(L)}, \text{KV}_{\leq g+1}^{(L)})$. Thus, Equation (5) holds for $g+1$. By induction, the coupled procedures generate identical token sequences almost surely, implying that their induced distributions coincide. $\square$

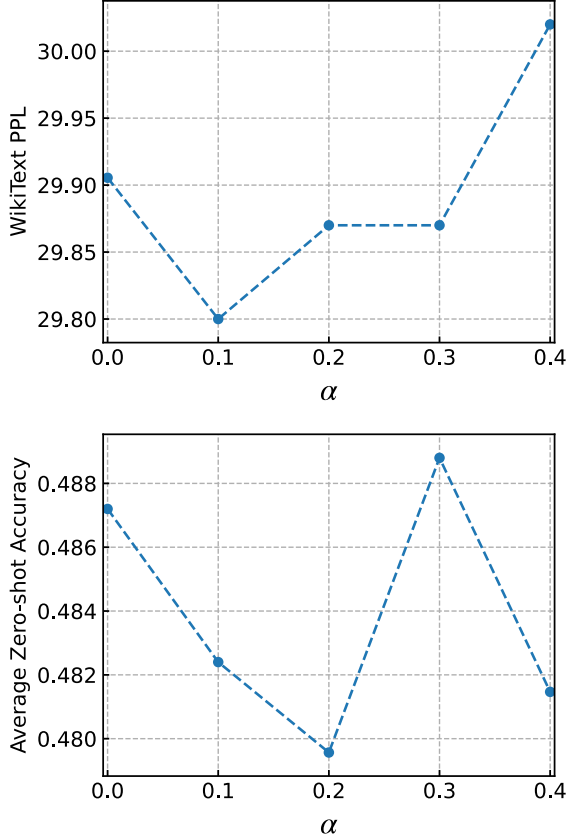


Figure 4: Ablations over α . Top: WikiText perplexity (PPL; lower is better). Bottom: Average zero-shot accuracy (higher is better), computed as the mean over ARC-Easy, HellaSwag, and SciQ.

Theorem A.6 shows that exactness requires equality only at the bottleneck interface $L \rightarrow L - 1$. No assumptions are needed about lower-level reconstructions to establish equality of the *output* distribution.

B Additional Experiments

B.1 Ablations over Strength of Recursive Loss

We ablate the hyperparameter α and evaluate its impact while keeping all other settings fixed, including the 4×4 block configuration. Figure 4 summarizes the results in terms of WikiText perplexity (PPL) and *average Zero-shot accuracy*, with the latter computed as the mean accuracy over ARC-Easy, HellaSwag, and SciQ. WikiText perplexity remains largely stable for $\alpha \leq 0.3$ and increases at $\alpha = 0.4$, indicating that an excessively strong recursive loss can negatively impact language modeling. Average zero-shot accuracy exhibits a non-monotonic trend: it decreases from $\alpha = 0.0$ to 0.2 , then recovers and peaks at $\alpha = 0.3$, be-

Table 3: Ablation study of chunk length. Memory usage per sample is reported in GiB, throughput, and TPM under PF and DE settings.

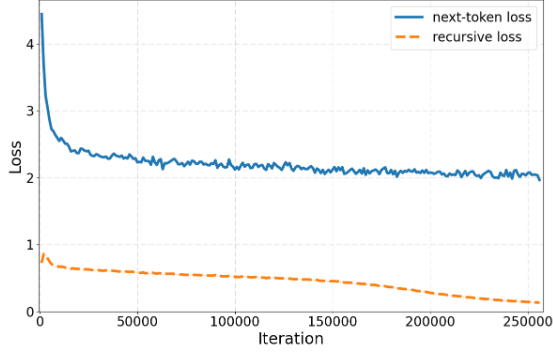
Metric	Setting	Chunk Length (C_1, C_2)	
		(2, 2)	(4, 4)
TPM	PF $\uparrow$	175.45	1262.58
	DE $\uparrow$	418.30	3062.17
Memory	PF $\downarrow$	0.066	0.031
	DE $\downarrow$	0.053	0.023
Throughput	PF $\uparrow$	11.58	39.14
	DE $\uparrow$	22.17	70.43
PPL $\downarrow$	-	24.28	29.91
Accuracy $\uparrow$	-	52.01	48.72

fore declining again at $\alpha = 0.4$. These results suggest that a moderate nonzero α can provide a slight downstream gain over $\alpha = 0.0$, but the benefit is sensitive to tuning and does not improve monotonically with α .

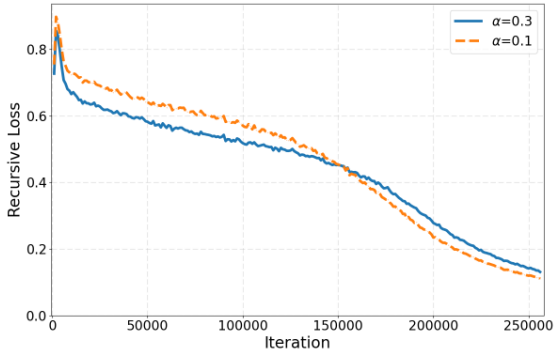
B.2 Ablations over Chunk Lengths

Chunk lengths $\{C_1, C_2\}$ directly determine the compression ratio of PHOTON’s hierarchical state and, the *effective context length* processed by the global (coarse) encoders. With a two-level hierarchy ($L = 2$) and a fixed token length T , the top-level sequence length is $M_2 = T/(C_1 C_2)$: smaller chunks yield longer latent streams and more frequent global updates, which can improve modeling fidelity while increasing global KV-cache traffic. To quantify this trade-off, we adhere to the primary setup outlined in Section 3, keeping all hyperparameters fixed (600M model, Pile pretraining, training context length of 2048, and the same evaluation protocol). We compare a finer hierarchy ($C_1=2, C_2=2; 2 \times 2$) to the default configuration ($C_1=4, C_2=4; 4 \times 4$).

Table 3 shows that 2×2 substantially improves quality, reducing WikiText PPL from 29.91 to 24.28 and increasing average zero-shot accuracy from 48.72 to 52.01, while incurring the expected cost in efficiency: under PF, throughput decreases from 39.14 to 11.58 K tok/s, and KV memory rises from 0.031 to 0.066 GiB (TPM drops from 1262.58 to 175.45); under DE, throughput decreases from 70.43 to 22.17 K tok/s, and memory increases from 0.023 to 0.053 GiB (TPM drops from 3062.17 to 418.30). These results highlight $\{C_1, C_2\}$ as a simple lever for navigating the TPM–quality Pareto



(a) **Next-token loss vs. recursive loss.** The recursive loss decreases steadily during training and becomes small in absolute magnitude. Although it is not directly comparable to the token-level decoding loss due to different units, its convergence to a low value indicates a small bottleneck mismatch.



(b) **Effect of α .** Recursive-loss trajectories for two values of α show that a smaller weight achieves a slightly lower loss late in training. In both cases, the loss continues to decrease, suggesting that extended optimization can further strengthen recursive consistency.

frontier: smaller chunks mitigate the compression bottleneck and increase the temporal resolution of the global latent stream, thereby improving perplexity and downstream accuracy, while larger chunks maximize throughput by minimizing global-state growth and KV traffic. Notably, even the quality-oriented 2×2 configuration remains significantly more memory-efficient than a vanilla Transformer at the same scale, as shown in Table 1, while also narrowing the quality gap. This suggests that PHOTON can flexibly convert some of its efficiency gains back into modeling performance without deviating from the Pareto frontier.

B.3 Recursive Consistency

Recursive Loss vs. Next-Token Loss. RecGen replaces bottom-up re-encoding with summaries computed from the decoder-side bottleneck reconstruction, making *recursive consistency* between encoder states and top-down reconstructions critical. A practical indicator of this consistency is

the recursive loss in Eq. 1. Figure 5a reports training dynamics for the 600M PHOTON model with $\alpha = 0.3$ ($(C_1, C_2) = (4, 4)$): the recursive loss steadily decreases and becomes small in absolute value by the end of training. Although the token-level negative log-likelihood and the cosine-distance-based recursive loss are not directly comparable in scale or units, the diminishing recursive term suggests that the top-down pathway can closely approximate the encoder-side bottleneck state. This provides empirical support that RecGen should only minimally perturb generation behavior when the bottleneck mismatch is sufficiently small. We also find that the recursive loss decreases even when $\alpha = 0$, indicating that the hierarchical architecture itself may encourage self-consistent summaries; a more detailed characterization of how residual mismatch translates into long-horizon distributional drift under RecGen is left for future work

Evaluating Likelihood of RecGen. RecGen updates the global state using model-generated reconstructions instead of ground-truth tokens, making exact likelihood evaluation under the *RecGen-induced* process difficult to incorporate into standard teacher-forcing pipelines. As a result, computing perplexity and zero-shot accuracy *under RecGen* in a fully consistent likelihood-based protocol is non-trivial. We therefore report perplexity and zero-shot results under standard teacher-forced execution, i.e., HierGen, and leave a more rigorous likelihood-based evaluation of RecGen for future work.

Dependence on Reconstruction Weight α . Figure 5b compares the recursive-loss trajectories of the 600M PHOTON model with $\alpha \in \{0.3, 0.1\}$. A smaller weight results in a slightly lower recursive loss late in training, and in both cases, the loss continues to decrease until the end of optimization. These trends suggest that recursive consistency can be improved not only by tuning the auxiliary objective but also by extending the training duration, which may further narrow the gap between RecGen and the encoder-consistent decoding procedure.

C Additional Implementation Details

C.1 Training Setting

We employ the 134B Pile-uncopyrighted dataset, consisting of 177,008,913 documents and 134,217,728,000 tokens (Gao et al., 2020). We

also utilize the Llama tokenizer, which has a vocabulary size of $\#(\mathcal{V}) = 32,000$. For training, we set the total batch size to 256, the context window to 2048, and the number of training epochs to 1 for PHOTON. For both Vanilla Transformer and Block Transformer, we align the training compute budget with the corresponding PHOTON model in terms of total Flops. We employ the Adam optimizer with a learning rate of 3×10^{-4} and a warm-up period of 3,000 steps. The scalar hyperparameters of PHOTON are set to $\beta = 0.0$. We further ablate the strength of the recursive loss by sweeping β , see Appendix B.1; increasing β generally improves downstream zero-shot performance. All experiments are conducted on an NVIDIA DGX H200 system.

C.2 Architecture

The vanilla Transformer employs an LLaMA architecture with adjusted parameter sizes. The configurations for the 600M, 900M, and 1.2B models are presented in Table 4, Table 5, and Table 6, respectively. Block Transformer follows the architecture proposed in the original paper with adjustment of parameters. The specifications for the 600M, 900M and 1.2B models are provided in Table 7, Table 8, and Table 9, respectively. The block decoders and token decoders in PHOTON are based on the LLaMA decoder architecture. The parameter configurations for the 600M, 900M, and 1.2B models are presented in Table 10, Table 11, and Table 12, respectively.

Table 4: Parameter breakdown for the Vanilla Transformer (600M).

Module	Hidden/ Int. / Layers	Params
Token Embedding (vocab=32000, d=1664)	1664 / - / -	53,248,000
Transformer Blocks (atten h=32, key value h=32, head d =52)	1664/4096/16	504,418,304
Final Norm (RMSNorm)	1664/ - / -	1,664
LM Head	1664/ - / -	53,248,000
Total	-	610,915,968

Table 5: Parameter breakdown for the Vanilla Transformer (900M).

Module	Hidden / Int. / Layers	Params
Token Embedding (vocab=32000, d=1792)	1792 / - / -	57,344,000
Transformer Blocks (atten h=32, key value h=32, head d =56)	1792/4608/20	752,424,960
Final Norm (RMSNorm)	1792/ - / -	1792
LM Head	1792/ - / -	57,344,000
Total	-	867,114,752

Table 6: Parameter breakdown for the Vanilla Transformer (1.2B).

Module	Hidden / Int. / Layers	Params
Token Embedding (vocab=32000, d=1920)	1920 / - / -	61,440,000
Transformer Blocks (atten h=32, key value h=32, head d =60)	1920/5120/24	1,061,775,360
Final Norm (RMSNorm)	1920/ - / -	1,920
LM Head	1920/ - / -	61,440,000
Total	-	1,184,657,280

Table 7: Parameter breakdown for the Block Transformer (600M).

Module	Hidden / Int. / Layers	Params
Embedder (vocab=32000, d=416)	- / - / -	13,312,000
BlockDecoder (atten h=32, key value h=32, head d =52)	1664 / 4096 / 8	252,210,816
Ctx Converter (in d = 1664, out d = 1664)	- / - / -	5,541,120
Embedder (vocab=32000, d=1664)	- / - / -	53,248,000
TokenDecoder (atten h=32, key value h=32, head d =52)	1664 / 4096 / 8	252,210,816
LM Head	1664 / - / -	53,248,000
Total	-	629,770,752

Table 8: Parameter breakdown for the Block Transformer (900M).

Module	Hidden / Int. / Layers	Params
Embedder (vocab=32000, d=448)	- / - / -	14,336,000
BlockDecoder (atten h=32, key value h=32, head d =56)	1792/4608/10	376,214,272
Ctx Converter (in d = 1792, out d = 1792)	- / - / -	6,426,112
Embedder (vocab=32000, d=1792)	- / - / -	57,344,000
TokenDecoder (atten h=32, key value h=32, head d =56)	1792/4608/10	376,214,272
LM Head	1792 / - / -	57,344,000
Total	-	887,878,656

Table 9: Parameter breakdown for the Block Transformer (1.2B).

Module	Hidden / Int. / Layers	Params
Embedder (vocab=32000, d=480)	- / - / -	15,360,000
BlockDecoder (atten h=32, key value h=32, head d =60)	1920 / 5120 / 12	530,889,600
Ctx Converter (in d = 2048, out d = 2048)	- / - / -	7,376,640
Embedder (vocab=32000, d=1664)	- / - / -	61,440,000
TokenDecoder (atten h=32, key value h=32, head d =60)	1920 / 5120 / 12	530,889,600
LM Head	1920 / - / -	61,440,000
Total	-	1,207,395,840

Table 10: PHOTON (600M)

Level	Module	Hidden / Int. / Layers	Params
Enc. ($l = 1$)	Embedder (vocab=32000, d=416)	- / - / -	13,312,000
	Ctx Chunker (block=4, concatenate)	- / - / -	-
	Ctx Encoder (atten h=32, kv h=32, head d=52)	1664 / 4096 / 4	126,106,240
Enc. ($l = 2$)	Ctx Chunker (block=4, linear)	- / - / -	11,083,904
	Ctx Encoder (atten h=32, kv h=32, head d=52)	1664 / 4096 / 4	126,106,240
	Ctx Converter (in d=1664, out d=1664)	- / - / -	5,541,120
Dec. ($l = 2$)	Ctx Decoder (atten h=32, kv h=32, head d=52)	1664 / 4096 / 4	126,106,240
	Ctx Converter (in d = 1664, out d = 1664)	- / - / -	5,541,120
	Embedder (vocab=32000, d=1664)	- / - / -	53,248,000
Dec. ($l = 1$)	Ctx Decoder (atten h=32, kv h=32, head d=52)	1664 / 4096 / 4	126,106,240
	LM Head (in d=1664, out d=32000)	1664 / - / -	53,248,000
Total	-	-	646,399,104

Table 11: PHOTON (900M)

Level	Module	Hidden / Int. / Layers	Params
Enc. ($l = 1$)	Embedder (vocab=32000, d=448)	- / - / -	14,336,000
	Ctx Chunker (block=4, concatenate)	- / - / -	-
	Ctx Encoder (atten h=32, kv h=32, head d=56)	1792 / 4608 / 5	188,108,032
Enc. ($l = 2$)	Ctx Chunker (block=4, linear)	- / - / -	12,854,016
	Ctx Encoder (atten h=32, kv h=32, head d=56)	1792 / 4608 / 5	188,108,032
	Ctx Converter (in d=1792, out d=1792)	- / - / -	6,426,112
Dec. ($l = 2$)	Ctx Decoder (atten h=32, kv h=32, head d=56)	1792 / 4608 / 5	188,108,032
	Ctx Converter (in d = 1792, out d = 1792)	- / - / -	6,426,112
	Embedder (vocab=32000, d=1792)	- / - / -	57,344,000
Dec. ($l = 1$)	Ctx Decoder (atten h=32, kv h=32, head d=56)	1792 / 4608 / 5	188,108,032
	LM Head (in d=1792, out d=32000)	1792 / - / -	57,344,000
Total	-	-	907,162,368

Table 12: PHOTON (1.2B)

Level	Module	Hidden / Int. / Layers	Params
Enc. ($l = 1$)	Embedder (vocab=32000, d=480)	– / – / –	15,360,000
	Ctx Chunker (block=4, concatenate)	– / – / –	–
	Ctx Encoder (atten h=32, kv h=32, head d=60)	1920 / 5120 / 6	265,445,760
Enc. ($l = 2$)	Ctx Chunker (block=4, linear)	– / – / –	14,755,200
	Ctx Encoder (atten h=32, kv h=32, head d=60)	1920 / 5120 / 6	265,445,760
	Ctx Converter (in d=9728, out d=2432)	– / – / –	7,376,640
Dec. ($l = 2$)	Ctx Decoder (atten h=32, kv h=32, head d=60)	1920 / 5120 / 6	265,445,760
	Ctx Converter (in d = 2432, out d = 2432)	– / – / –	7,376,640
	Embedder (vocab=32000, d=1920)	– / – / –	61,440,000
Dec. ($l = 1$)	Ctx Decoder (atten h=32, kv h=32, head d=60)	1920 / 5120 / 6	265,445,760
	LM Head (in d=1920, out d=32000)	1920 / – / –	61,440,000
Total	–	–	1,229,531,520